COMP 3271 Project Work: Getting Started

Overview

The project work in this course focuses upon the design and development of network applications using the C programming language. The project work in COMP 3271 is worth 28% of your total grade. There are four parts to the project:

- Project Part 1: C Programming Language and Sockets (6%)
- Project Part 2: Basic TCP/IP Socket Programming (6%)
- Project Part 3: Programming at the Data Link Layer (10%)
- Project Part 4: A Basic File Transfer Application (6%)

The C Programming Language

Each part of the project work will introduce you to a different aspect of network programming. Because most network programs require a fast processing language, they use C programming languages. For this reason, the C programming language is used in this project.

It is assumed that you know Java programming language, but you may not have used C programming language before. For that reason, we begin the project work in this course with two introductory exercises. These coding exercises, developing *welcome.c* and *file_copy.c*, are described later in this document. The purpose of these exercises is to provide you with some experience and comfort in using the C programming language.

References – The C Programming Language

Kerninghan, B., & Ritchie, D. (1988). *The C programming language* (2nd ed.) [Digital book]. New Jersey: Prentice Hall Software Series. Retrieved from http://mef-lab.com/osnove-2016/C-Programming-Ebook.pdf

Tutorials Point. (2018). *C tutorial* [Educational tutorial]. Retrieved from https://www.tutorialspoint.com/cprogramming/index.htm

Student Programming Environment

For the project work, you will need to use Linux.

- For Mac users, terminal windows can be used. You may need to download and install the gcc compiler.

- For Linux users, terminal windows can be used. The gcc compiler should already be installed.
- For Windows users, you will need to install the Linux operating system, e.g., Ubuntu. There are three options:
  i. If you are running Windows 10, you may install Ubuntu for Windows 10, which is a free download from the Microsoft Store.
     - See the detailed description of the procedure for installing Ubuntu for Windows 10.
     - Once Ubuntu is installed, you should update the system: ***$ sudo apt-get update***. Next, install the gcc compiler and associated libraries: ***$ sudo apt install gcc***.
     - Verify that the gcc compiler is installed: ***$ gcc –version***.

     Your Windows Subsystem for Linux is now ready for use.
  ii. You may also install a Ubuntu Linux system as a standalone operating system and set up the dual-boot option that will allow you, at boot time, to decide which operating system you want to use. This is complex and not generally recommended.
  iii. It is recommended that students install a virtual machine called VirtualBox. VirtualBox is a free and powerful virtual machine that allows an unmodified operating system with all its installed software to run in a special environment, on top of your existing operating system. Refer to the document entitled "How to Install Ubuntu on VirtualBox" for detailed installation instructions for VirtualBox and Ubuntu.

Integrated Development Environment

You do not have to use any integrated development environment (IDE) or debugger in this course. It is suggested that students use a text editor, e.g., nano or vi, to make programs.

A Brief Comparision of C to Java

Pros:

- Fast execution
- Easy to handle memory ⇨ Good for system programming
- Bit operation

Cons:

- Complex concepts of pointer, type conversion, and memory allocation

Note: The project in this course does not rely much on those complex concepts. For the most part, **we will use common syntaxes of C and Java.**

Differences between C and Java

C	Java
Procedural: • No class • Common data: global variables • Abstract data type: struct, union	Object-oriented: • Class • Common data: instance variables • Abstract data type: class
Micro approach: • Individual utility libraries	Macro approach: • Utilities included in language itself
Reference type variable: • Pointer	Generally no reference, but objects include the concept
Call by value, call by reference	Call by value mostly, call by reference for objects
Compiling: one file at a time; linking	Compiling: cross-reference

Basic data types in C

- `char` `unsigned char` `// unsigned variables use the left most bit`
- `short` `unsigned short` `// which is used as a sign (+/-) in signed variables`
- `int` `unsigned int`
- `long` `unsigned long`
- `float` `unsigned float`
- `double` `unsigned double`
- **There is no Boolean type in C. Any non-zero value is considered as TRUE, and zero value is considered as FALSE.**

- Array data type for any data type
- Pointer data type for any data type: E.g., `int*`, `char*`, …

Operators in C

The basic operators you will need include the following:

- `+, -, *, /, ++, --, >, <, >=, <=, =, ==, !=, &&, ||,` ...

operand1 **&** operand2	bitwise AND operation
operand1 **|** operand2	bitwise OR operation
operand1 **^** operand2	bitwise XOR operation
operand1 **>>** *n*	shift rightward; the *n* leftmost bit[s] will be filled with 0
operand1 **<<** *n*	shift leftward; the *n* rightmost bit[s] will be filled with 0
~operand	bitwise NOT operation; one's complement
`&`operand	the address of the first byte for the variable in the main memory
`*`operand	pointer to the address stored in the operand, i.e., the value stored in the address

- Implicit and explicit type conversion

Header Files in C

- Include constants and data types
- For example, `/usr/include/stdio.h` includes constants and data types that are used for functions in the standard input/output library.
- In a program,
  - `#include <stdio.h>  /* /usr/include/stdio.h */.`
- `<...>` is searched from `/usr/include`.
- `"..."` is searched from the current working directory.

Note: The include statement is like the import statement in Java.

The main() Function in C

There should be one main function like main method in Java programs.

```
int main(int argc, char* argv[])
```

`argc` contains the number of arguments that are given to the program when it is executed, including the name of the program itself

argv[] array of `char*`

`char*` points to an address which contains a **string** that is a list of characters ending with `'\0'` character; this is very similar to a **`char` array**

`argv[0],...` are strings for the argument on the command line when the program is executed (e.g., `$ nano welcome.c`)

Input and Output in C

If you retrieve the manual for `printf`, e.g., `$ man -S 3 printf`,

```
#include <stdio.h>      // necessary header file for printf()
int printf(const char *format, …)
```

format includes all information how to print; `%d, %c, %f, %s, %u, %o, %x, …`

For example:

```
printf("%d %c %f %u %s", i, c, x, &x, buf);
// int i, char c, float x, address of x, char* buf or char buf[]
getchar()          a character from stdin
getline()          a line from a FILE
```

Note: We will not use these functions in this course if possible.

For low-level file input and output, use the following:

```
open()
int open(...);                // returns the file descriptor
read()
int read(int fd, char* buf, int length);
write()
int write(int fd, char* buf, int length);
close()
int close(fd);                // close the file referenced by fd
```

Note: `read()` and `write()` take three parameters—the file descriptor, the buffer containing data `(buf)`, and the number of bytes to be transferred. Low-level I/O functions return a negative value when there is an error.

Predefined file descriptors—no need to open again; already opened

`0`	standard input
`1`	standard output
`2`	standard error

Global Variables

- Variable declared not inside a function
- Common data for all functions in the file
- It is like public instance variables in Java.
- In order to access a global variable declared in other file,
  - `extern data_type variable_name;`

User-Defined Data Type

- **define** – macro and constant definition; like final variable declaration in Java

```
#define     BUF_SIZE    512
```

- **struct** – it is like a class definition in Java, which has only instant variables

```
struct record {
        int   number;
        char  name;
};

struct record kildong_record;
```

- **union**

```
union weight {
        int   kilo;
        int   pound;
};

union weight kildong_weight;
```

- **typedef**

```
typedef struct record record;
record kildong_record;
```

Note: We will not use union and typedef in this course if possible.

Linking and Libraries

- `$ gcc -c main.c` compile `main.c` into `main.o`
- `$ gcc main.o student.o -o test -lm` `-lm` means library `libm.a` that is in `/usr/lib/`

Handling Strings

A string in C is a consecutive list of bytes (or characters), usually in an array, ending with `'\0'` (0 value). When we use general data, do not use these functions because even `'\0'` can be a part of the data.

- `<string.h>` related header file
- **`strcpy()`** string copy; e.g., `strcpy(dest_str, src_str);` // two arguments could be `char` arrays
  - **`strcpy(buf, "Welcome to ...\n");`**
    // buf is a char array of enough room
- **`strlen()`** string length; e.g., len = strlen(argv[0]);
- `strcat()` string concatenation; e.g., `strcat(dest_str, src_str);`
- `strcmp()` string compare; e.g., `i = strcmp(first_str, second_str);`
- **`atoi()`** convert to integer; e.g., `i = atoi(argv[2]);`
- `atof()` convert to float; e.g., `f = atof(argv[1]);`

Handling Memory

- `malloc()` allocate dynamic memory
- `free()` purge an allocated dynamic memory

1. Exercise Program: *`welcome.c`*

Each program consists of:

- include, ; import in Java
- define for constants and ; final variable declaration for constants in Java
- global variable declaration statements ; instance variables in Java
- main and other functions ; methods

```
// it is like import statement in Java
#include <stdio.h>      // standard i/o; this include statement is like
import statement in Java.
#include <stdlib.h>     // standard library


#define SIZE 128        // constant definition; it is like final
variable declaration in Java.


int test = 5;           // global variable; it is like a public
instance variable in Java.


int main(int argc, char* argv[])    // or char **argv
                                    // char* may be used for strings.
                                    // argc - # of arguments including
a.out
{
      int   i;
      char  name[32];


      printf("Welcome to C programming world!\n");


      printf("%d arguments:\n", argc);    /* %d means an integer
            */
      for (i = 0; i < argc; i++)
            printf("\t%s\n", argv[i]);    // %s means a string from the
next argument
      name[0] = 'C'; name[1] = 'o'; name[2] = 'm'; name[3] = 'p';
name[4] = 'u';
name[5] = 't'; name[6] = 'e'; name[7] = 'e'; name[8] = '\0'; name[9] =
'r';
printf("The length of %s is %d.\n", name, strlen(name));    // what
will be printed?
}
```

Compiling

To compile the program, you will use the following statements:

```
$ gcc welcome.c               or    // it will create a.out
$ gcc welcome.c –o welcome          // it will create welcome instead
of a.out
```

Running

To run the program, use the following:

```
$ ./a.out
$ ./welcome
```

2. Exercise Program: *file_copy.c*

```
/*
 * copy a file to another file
 */


#include <stdio.h>
#include <stdlib.h>


#include <sys/types.h>      // These four header files are necessary to
use file i/o functions.
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>


/* argv[1]: from */
/* argv[2]: to */


int main(int argc, char* argv[])
{
              int in, out;


              in = open(argv[1], O_RDONLY);            // O_RDONLY is
              defined in /usr/include/sys/types.h
              if (in < 0) {
                        printf("Cannot open the file %s\n", argv[1]);
                        exit(1);
              }


              out = open(argv[2], O_WRONLY | O_TRUNC | O_CREAT, S_IRUSR
              | S_IWUSR);
              if (out < 0) {
                        printf("Cannot create the file %s\n", argv[2]);
```

```
                exit(1);
            }

            ... // read some bytes from in and write them into out.
            ... // print those bytes onto the screen while copying.

            close(in);
            close(out);

            ... // test if the two files have the exactly same
            contents, and print the result
}
```

Compiling

To compile the program, you will use the following statements:

```
$ gcc file_copy.c –o file_copy
```

Running

To run the program, use the following:

```
$ ./file_copy
```